

# 조향센서 없이 MG996R 조향각을 보정한 트러블슈팅 기록

← 프로젝트 전체 트러블슈팅 README

## 1. 문제 상황

최종 차량에 MG996R 조향 서보는 장착했지만 실제 바퀴 조향각을 읽을 AS5600/PP-A818 센서는 사용할 수 없는 상태였다. 따라서 STM32는 다음 두 값을 직접 알 수 없었다.

- 서보 PWM에 따라 실제 좌우 앞바퀴가 몇 도 회전했는지
- 주행 중 충격, 기구 유격 또는 타이어 하중 때문에 명령각과 실제 각도가 달라졌는지

조향센서가 없는 상태에서 단순히 750~1680 us를 일정한 각도로 나누면 서보혼과 링크의 비선형성, 좌우 비대칭 및 안티 아커만 특성을 반영할 수 없다. 이 문제를 해결하기 위해 PWM별 좌우 바퀴각을 수동 측정하고, 차량 중심 기준의 등가 조향각을 계산한 뒤 7점 LUT로 저장했다.

이 방식은 실제 센서 피드백을 대체하는 페루프 제어가 아니다. 현재 차량의 정적인 명령 조향각 ↔ PWM 관계를 실측값으로 근사하는 방법이다.

## 2. 하드웨어 및 좌표 규칙

| 항목              | 확정값                          |
|-----------------|------------------------------|
| 서보 PWM          | STM32 PB4 / TIM3_CH1 / 50 Hz |
| 서보 전원           | 외부 5 V 강압 전원, STM32와 공통 GND  |
| 왼쪽 끝            | 750 us                       |
| 직진              | 1215 us                      |
| 오른쪽 끝           | 1680 us                      |
| 휠베이스 L          | 135 mm (0.135 m)             |
| 앞 조향축 중심 간 거리 T | 85 mm (0.085 m)              |

차량을 위에서 보고 차량 진행 방향을 기준으로 부호를 정했다.

+조향각: 차량 왼쪽 / 반시계 방향  
0도:     차체 중앙선과 평행  
-조향각: 차량 오른쪽 / 시계 방향

좌회전에서는 양쪽 바퀴각이 모두 양수이고, 우회전에서는 모두 음수다. 안쪽/바깥쪽 바퀴는 부호가 아니라 각도 크기로 구분한다.

## 3. 측정 중 발견한 자동 중앙복귀 문제

초기 `servo-calibrate` 명령은 지정한 펄스를 일정 시간 전송한 뒤 `0 us` 종료 명령을 보냈다. 하지만 STM32 `ControlTask`가 진단 모드를 종료한 직후 일반 조향 명령 `0도`를 다시 적용하면서 서보가 `1215 us` 중앙으로 자동 복귀했다.

진단 펄스 종료  
 → TIM3 PWM 정지  
 → 일반 조향 제어 재개  
 → 목표각 `0도`  
 → `1215 us` 출력  
 → 바퀴 중앙복귀

이 상태에서는 펄스 위치에 선을 긋고 각도를 측정하기 어려웠다. 진단 종료 `0 us`는 진짜 PWM OFF로 처리하고, 다음 명령이 들어오기 전까지 중앙 명령을 재출력하지 않도록 `task_control.c`를 수정했다.

주의할 점은 PWM을 끄면 서보 토크도 사라진다는 것이다. 소프트웨어가 중앙으로 복귀시키지는 않지만 링크의 탄성이 나 외력 때문에 바퀴가 움직일 수 있다.

## 4. 수동 측정값

각 PWM에서 차체 진행 방향을 `0도`로 놓고 왼쪽 앞바퀴와 오른쪽 앞바퀴의 방향을 각각 측정했다.

| PWM  | 왼쪽 바퀴각 | 오른쪽 바퀴각 |
|------|--------|---------|
| 750  | +15도   | +22도    |
| 905  | +10도   | +17도    |
| 1060 | +8도    | +12도    |
| 1215 | 0도     | 0도      |
| 1370 | -8도    | -6도     |
| 1525 | -17도   | -18도    |
| 1680 | -29도   | -26도    |

PWM이 증가할수록 **왼쪽 → 직진 → 오른쪽**으로 단조롭게 변하므로 LUT 입력으로 사용할 수 있다. 다만 좌우 바퀴각 차이와 좌우 최대각 비대칭이 크기 때문에 전체 범위를 하나의 `us/degree` 상수로 환산하면 안 된다.

## 5. 좌우 바퀴각을 등가 중심각으로 변환

왼쪽 바퀴의 횡방향 위치를 `+T/2`, 오른쪽을 `-T/2`로 두었다. 각 바퀴가 요구하는 곡률은 다음과 같이 계산했다.

```
k_left  = tan(delta_left) / (L + (T/2) * tan(delta_left))
k_right = tan(delta_right) / (L - (T/2) * tan(delta_right))

k_equivalent = (k_left + k_right) / 2
delta_center = atan(L * k_equivalent)
```

실제 측정값은 완전한 아커만 조건을 만족하지 않으므로 좌우 바퀴가 요구하는 회전 중심이 서로 다르다. 이 경우 실제 주행에서는 타이어 미끄러짐과 기구 변형이 발생하므로, 초기 명령 기반 오도메트리에 사용할 값으로 좌우 곡률의 평균을 사용한 등가 중심각을 선택했다.

| PWM  | 왼쪽/오른쪽 바퀴각 | 안티 아커만 | 곡률 불일치 | 등가 중심각  |
|------|------------|--------|--------|---------|
| 750  | +15 / +22도 | +37.8% | 60.8%  | +19.55도 |
| 905  | +10 / +17도 | +51.9% | 67.8%  | +14.18도 |
| 1060 | +8 / +12도  | +40.0% | 51.4%  | +10.27도 |
| 1215 | 0 / 0도     | 0%     | 0%     | 0도      |
| 1370 | -8 / -6도   | +28.6% | 36.4%  | -7.09도  |
| 1525 | -17 / -18도 | -5.7%  | 13.7%  | -17.56도 |
| 1680 | -29 / -26도 | +10.9% | 45.4%  | -28.69도 |

양수 안티 아커만은 바깥쪽 바퀴가 안쪽 바퀴보다 더 많이 꺾였다는 뜻이다. 이 수치는 분석용이며 STM32 제어 명령에 직접 사용하지 않는다.

## 6. 최종 7점 조향 LUT

STM32에는 좌우 바퀴각 원본 대신 등가 중심각 ↔ PWM을 저장한다.

| 등가 중심각  | PWM     |
|---------|---------|
| +19.55도 | 750 us  |
| +14.18도 | 905 us  |
| +10.27도 | 1060 us |
| 0도      | 1215 us |
| -7.09도  | 1370 us |
| -17.56도 | 1525 us |
| -28.69도 | 1680 us |

현재 첫 통합 주행의 운영 명령 범위는  $\pm 10$ 도로 제한한다. LUT 자체는 전체 측정 범위를 지원하지만 더 큰 각도는 바퀴 저속 회전반경을 확인한 뒤 개방한다.

## 7. 측정하지 않은 빈 각도의 PWM 계산

측정한 7개 각도만 사용할 수 있는 것이 아니다. 목표각을 포함하는 인접한 두 LUT 지점을 찾고 그 구간 안에서 비례식, 즉 구간별 선형 보간을 수행한다.

두 측정점을 ( $angle\_0$ ,  $pulse\_0$ ), ( $angle\_1$ ,  $pulse\_1$ )이라고 하면:

$$ratio = (target\_angle - angle\_0) / (angle\_1 - angle\_0)$$

```
target_pulse =
    pulse_0 + ratio * (pulse_1 - pulse_0)
```

계산 결과는 TIM3이 사용하는 정수 마이크로초 단위로 반올림한다.

### 예시 1: 목표각 +5도

+5도는 0도 ↔ +10.27도 구간에 있다.

```
0도      → 1215 us
+10.27도 → 1060 us

ratio = 5 / 10.27
      = 약 0.487

pulse = 1215 + 0.487 * (1060 - 1215)
      = 약 1139.5 us
      → 1140 us
```

### 예시 2: 목표각 -10도

-10도는 -7.09도 ↔ -17.56도 구간에 있다.

```
-7.09도 → 1370 us
-17.56도 → 1525 us

ratio = (-10 - -7.09) / (-17.56 - -7.09)
      = 약 0.278

pulse = 1370 + 0.278 * (1525 - 1370)
      = 약 1413 us
```

검증된 중간값 예시는 다음과 같다.

| 목표 중심각 | 계산 PWM  |
|--------|---------|
| +10도   | 1064 us |
| +5도    | 1140 us |
| 0도     | 1215 us |
| -5도    | 1324 us |
| -10도   | 1413 us |

전체 범위에 하나의 비례식을 사용하지 않는 이유는 각 구간의 각도 변화량이 서로 다르기 때문이다. 각 인접 지점마다 별도의 기울기를 사용하는 구간별 보간이 실제 링크의 비선형성을 더 잘 반영한다.

측정 범위를 벗어난 목표각은 외삽하지 않는다.

목표각  $\geq +19.55^\circ \rightarrow 750 \text{ us}$   
 목표각  $\leq -28.69^\circ \rightarrow 1680 \text{ us}$

## 8. 실제 코드 반영 위치

| 파일                                             | 역할                                              |
|------------------------------------------------|-------------------------------------------------|
| <code>App/Inc/vehicle_config.h</code>          | 7개 등가 중심각/PWM 및 차량 형상값 저장                       |
| <code>App/Src/task_control.c</code>            | 부팅 시 7점 LUT 등록, 진단 종료 상태 처리                     |
| <code>Drivers/BSP/Inc/steering_sensor.h</code> | <code>SteeringServoLutPoint</code> 와 LUT API 선언 |
| <code>Drivers/BSP/Src/steering_sensor.c</code> | LUT 검증, 구간 탐색, 선형 보간, 끝값 제한                     |

기존 3점 보정 API는 다른 진단 코드와의 호환성을 위해 유지했고, 실제 차량 부팅 설정은 7점 LUT API를 사용한다.

현재 코드 반영 범위는 목표 등가 중심각  $\rightarrow$  서보 PWM 변환까지다. `OdometryTask`는 아직 물리 조향센서의 유효값을 요구하므로 위치 적분이 비활성화되어 있다. 다음 통합 단계에서 명령 등가각을 추정 입력으로 연결하고, 텔레메트리에 실제 센서값이 아니라 추정값임을 표시해야 한다.

## 9. 확인 명령

실제 주행 명령 경로에서 빈 각도가 보간되는지 차량을 뒤집고 확인한다.

```
py tools\uart_protocol_test.py drive-test \
  --port COM11 \
  --speed-mm-s 0 \
  --steering-deg 5 \
  --seconds 1.5 \
  --wheels-off-ground
```

+5도는 왼쪽 조향이며 계산상 약 1140 us가 적용된다.

```
py tools\uart_protocol_test.py drive-test \
  --port COM11 \
  --speed-mm-s 0 \
  --steering-deg -5 \
  --seconds 1.5 \
  --wheels-off-ground
```

-5도는 오른쪽 조향이며 계산상 약 1324 us가 적용된다.

## 10. 얻은 교훈과 한계

1. 서보 데이터시트의 일반적인 PWM 범위만으로 실제 바퀴각을 정할 수 없다.
2. 서보혼 각도보다 최종 좌우 바퀴각을 직접 측정해야 차량 곡률을 추정할 수 있다.
3. 좌우 링크가 비대칭이면 하나의 `us/degree` 값보다 구간별 LUT가 적합하다.
4. 진단 종료 후 일반 제어가 다시 PWM을 출력하는지 상태 전환까지 확인해야 진짜 PWM OFF를 보장할 수 있다.
5. 센서 없는 LUT는 명령값 추정일 뿐 실제 조향각 피드백이 아니다.
6. 기구 유격, 노면 충격, 하중 및 타이어 미끄러짐은 LUT만으로 감지할 수 없다.
7. 최종 오도메트리 정확도는 바닥에서 직선거리와 회전반경을 실측해 확인해야 한다.
8. IMU가 추가되면 아커만 계산 yaw와 IMU yaw rate를 비교해 미끄러짐과 누적 방향 오차를 보정할 수 있다.

이번 방법으로 조향센서가 없어도 저속 주행 명령과 초기 아커만 오도메트리를 구현할 수 있지만, 실제 센서가 추가된다면 LUT는 목표 PWM의 초기값으로 사용하고 센서 피드백으로 실제 각도를 검증하는 구조가 더 정확하다.